

파일 권한 및 umask

✎ 추가 일시	2026년 9월 7일 오후 8:32
⚙ 상태	시작 전

파일 권한 및 umask

왜 파일 권한이 필요한가

파일시스템 ⇒ 여러 사용자와 여러 프로세스가 함께 쓰는 공유 자원

- 권한이 없다면 어떤 프로세스든 아무 파일이나 읽고 쓰고 지울 수 있게 됨
- 이는 사용자 한 명의 실수나 악성 코드 하나가 시스템 전체(다른 사람 데이터, 커널 설정, 인증 정보 등...)을 망가뜨릴 수 있다는 뜻

권한⇒ 이러한 경계를 파일 시스템 수준의 코드로 표현한 것

이건 최소 권한 원칙(principle of least privilege)의 파일 시스템 버전

- 누구든 필요한 만큼의 권한을 갖게하고 그 이상은 주지 않게 하는 것

리눅스에서 프로세스는 그것을 실행한 사용자의 권한을 그대로 물려받음

그래서 어떤 파일에 누가 어떤 권한을 갖는다 = 이 프로세스가 이 자원에 대해 할 수 있는 일의 경계

개념 정리

표준 권한 (ugo / rwx)

대상은 소유자(u) / 소유 그룹(g) / 그 외(o) 셋, 권한은 읽기(r=4) / 쓰기(w=2) / 실행(x=1)

파일과 디렉터리에서 같은 문자(rwx)가 다르게 작동함

권한	파일에서	디렉터리에서
r	내용 읽기	ls 로 목록 조회
w	내용 수정	파일 생성 및 삭제/ 이름 변경
x	실행	cd 진입 / 경로 통과

주의할 부분: 디렉터리에 `r` 만 있고 `x` 가 없으면 안의 파일 이름은 `ls` 로 보여도 실제 접근(`cat`, `cd`)은 막힘!!

파일의 이름과 권한은 서로 다른 곳에 저장

- 실제 데이터/소유자/권한 비트는 inode에 존재
- 디렉터리는 "이름 → inode 번호"라는 매핑 목록(엔트리)만 가진다
 - 디렉터리에 대한 `w` 권한은 그 안 파일의 내용을 고칠 권한이 아닌 그 디렉터리의 엔트리 목록을 고칠 권한
 - 파일을 만들고 지우고 이름을 바꾸는 행위는 그 파일이 속한 디렉터리의 쓰기 권한으로 결정
 - ⇒ 파일 자체의 권한과는 무관~~

주의할 부분: 어떤 파일이 `chmod 000 ->` 그 파일이 있는 디렉터리에 쓰기 권한만 있으면 삭제 가능

⇒ 삭제 가능 여부는 파일 자체 권한이 아니라 부모 디렉터리 권한이 결정하기 때문

chmod

```
chmod u+x file    # 소유자에 실행 권한 추가
chmod g-w file    # 그룹 쓰기 권한 제거
chmod o=r file    # 기타는 읽기만
chmod 755 file    # rwxr-x
```

rwx 세 자리는 실제로 3비트 이진수

- r는 100(=4), w는 010(=2), x는 001(=1)
- 이 셋을 OR로 합친 값
 - `rwx` = 111=7, `rw-` = 110=6, `r-x` = 101=5
 - 0~7의 범위를 가짐

특수 권한 — SUID / SGID / Sticky bit

기본 규칙:

- 프로그램을 실행하면 그 프로세스는 실행한 사람의 권한으로 동작
- 새 파일을 만들면 소유자=만든 사람/소유 그룹=만든 사람의 기본 그룹
- 아래 셋은 바꾸는 예외 규칙들

권한	숫자	개념
SUID	4	실행 파일에 설정 — 실행되는 동안 프로세스 신분이 실행한 사람 → 파일 소유자 로 바뀐다
SGID	2	디렉터리에 설정 — 그 안에서 새로 생성되는 파일의 소유 그룹이 생성자의 기본 그룹 → 디렉터리 자신의 그룹 으로 바뀐다 (실행 파일에 걸면 SUID와 같은 방식으로 그룹 신분이 바뀜)
Sticky bit	1	디렉터리에 설정 — 그 안 파일의 삭제 및이름 변경 권한이 디렉터리에 쓰기 권한 있는 모두 → 그 파일의 소유자와 root 만으로 좁혀진다

SUID —

비밀번호 변경은 `/etc/shadow` (root만 쓰기 가능)를 고쳐야함

하지만 일반 사용자도 자기 비밀번호는 바꿀 수 있어야 하는데

`/usr/bin/passwd` 에 SUID가 걸려 있어서 누가 실행하든 실행되는 동안만 root 신분으로 동작

(passwd가)해 shadow를 고칠 수 있고 끝나면 원래 권한으로 돌아감

```
ls -l /usr/bin/passwd
-rwsr-xr-x. 1 root root ... /usr/bin/passwd
  ↑ 소유자 실행 자리가 x 대신 s
```

SGID —

kim, lee가 `/shared/team` 을 같이 쓴다고 하면...

UPG 방식 때문에 기본 그룹이 각자 kim, lee로 다름

그냥 두면 kim이 만든 파일 그룹은 kim/ lee가 만든 파일 그룹은 lee가 되어

서로 그룹 권한으로 상대 파일을 못 건드림;;

디렉터리 그룹을 dev로 맞추고

- `usermod -aG dev lee` , `usermod -aG dev kim`
- 를 통해 dev라는 그룹에 lee와 kim을 임시 추가해줌
- `-G dev` 이 사용자의 보조(secondary) 그룹을 dev로 설정
 - `-a` 는 기존그룹은 그대로 두게 하는것

SGID를 걸면 누가 만들든 새 파일의 그룹이 자동으로 dev가 됨

```
ls -ld /shared/team
drwxrwsr-x. 2 root dev ... /shared/team
↑ 그룹 실행 자리가 x 대신 s
```

Sticky bit —

`/tmp` 는 누구나 파일을 만들어야 해서 전체 쓰기 권한(777)을 줌

그러면 원칙적으로 남의 임시 파일도 아무나 지울 수 있음 $\pi\pi$

sticky bit는 "디렉터리 쓰기 권한이 있어도 그 안 파일은 소유자와 root만 지우거나 이름을 바꿀 수 있다"는 예외를 추가할 수 있게함

```
ls -ld /tmp
drwxrwxrwt. ... /tmp
↑ 기타(other) 실행 자리가 x 대신 t
```

부여 및 해제 방법

755 처럼 3자리로 쓰는 건 생략형이고, 정식으로는 맨 앞에 특수 권한 전용 자리가 하나 더 있음

(SUID=4, SGID=2, sticky=1이 이 4 번째 자리를 생략하면 0으로 취급)

심볼릭 방식: 기존 rwx 조작과 동일한 문법

숫자 방식: ugo 3자리 앞에 붙이기

```
chmod u+s file      # SUID 추가 (숫자: chmod 4755 file)
chmod g+s dir       # SGID 추가 (숫자: chmod 2775 dir)
chmod +t dir        # sticky 추가 (숫자: chmod 1777 dir)
chmod 6755 file     # SUID(4)+SGID(2) 동시 적용도 값을 더해서 가능

# 해제
chmod u-s file
chmod g-s dir
chmod -t dir
chmod 0755 file     # 숫자 방식은 앞자리를 0으로(또는 생략)
```

대상 파일/디렉터리의 소유자이거나 root여야 바꿀 수 있음

소유권 변경 — chown / chgrp

```
chown kim:dev project/  # 소유자 및 그룹 동시 변경
chown :dev project/     # 그룹만 변경
```

```
chgrp dev project/          # 위와 동일한 효과
```

umask

앞으로 새로 만들어지는 파일 및 디렉터리의 **기본 최대 권한**에서 얼마를 뺄지 정하는 마스크

- chmod는 기존 파일에서 권한을 변경해주는 것이라 umask량은 좀 다름

기본 최대 파일 666(실행권한은 애초에 부여 안함) / 디렉터리 777

실제 생성 권한 = 기본 최대 권한 - umask

umask 022 → 파일 666-022=644(rw-r--r--)/ 디렉터리 777-022=755(rwxr-xr-x)

umask 027 → 파일 666-027=640(rw-r-----)/ 디렉터리 777-027=750(rwxr-x---

확인은 `umask` (8진수) 또는 `umask -S` (심볼릭)

세션에서 임시로 바꾸려면 그냥 `umask 027` 을 실행

영구 적용하려면 개인은 `~/.bashrc`

시스템 전체는 `/etc/bashrc` 또는 `/etc/profile` 에 넣어 적용

실제로는 **기본권한 AND (NOT umask)** 라는 비트 연산

파일 기본값 666 = rw-rw-rw- (애초에 실행 비트가 없음)

umask 111이라 가정해도 → 666 & ~111 = 666 (없는 비트는 뺄 것도 없음)

실습 문제

1. 숫자 모드로, `report.txt` 를 소유자는 읽기/쓰기, 그룹과 기타는 읽기만 가능하게 설정하라.

▼ 정답:

```
chmod 644 report.txt
```

2. `project/` 디렉터리의 소유자를 kim, 소유 그룹을 dev로 한 번에 바꿔라.

▼ 정답:

```
chown kim:dev project/
```

3. 여러 사용자가 함께 쓰는 `/shared/dropbox` 에서, 각자 자기 파일만 삭제할 수 있도록 설정하라.

▼ 정답:

```
chmod +t /shared/dropbox # 또는 chmod 1777
```